

CSCI 332, Fall 2026

Homework 5 (Part 2—Part 1 is on Prairie Learn)

Due Monday, September 28 Anywhere on Earth (6am Tuesday)

Submission Requirements

- Type or clearly hand-write your solutions into a PDF format so that they are legible and professional. Submit your PDF on Gradescope.
- Do not submit your first draft. Type or clearly re-write your solutions for your final submission. If your submission is not legible, we will ask you to resubmit.
- Use Gradescope to assign problems to the correct page(s) in your solution. If you do not do this correctly, we will ask you to resubmit.

Academic Integrity

Remember, you may access **any** resource in preparing your solution to the homework. However, you **must**

- write your solutions in your own words, and
- credit every resource you use (for example: “Bob Smith helped me on this problem. He took this course at UM in Fall 2020”; “I found a solution to a problem similar to this one in the lecture notes for a different course, found at this link: www.profzeno.com/agreatclass/lecture10”; “Here is a link to the full transcript of my chat with ChatGPT about the problems on this HW”.) Ask if you need any clarifications about how to properly cite your sources!

1. (0 points) List the resources you used for each problem in this homework. If you used no resources except lecture notes and videos and the linked textbook, you can say “none”. If you used an AI tool, you should put a link to your full conversation here, and you must use the [provided prompt](#) to guide the AI to act as a tutor, not an answer-provider.

You will be asked to resubmit your homework if this section is blank.

Remember that no matter what resources you consulted, you must write your solutions *in your own words*.

2. (5 points)

You work for the State of Montana’s Economic Development Office, and you have been tasked with analyzing income data from Montana’s households. However, the data source that you must use is a bit complicated. In order to save money, the state contracted with two different companies to collect income data in Montana’s two congressional districts, which we will assume each have exactly n households. We will also assume that no two Montana households have exactly the same income.

The two companies worked independently and use different databases to store their results. Each database stores income data for their respective n Montana households, so overall, there is data for $2n$ Montana households across these two databases. These databases support one query: given an integer k , they will return the k th smallest income.

The companies did the data collection for free, but now charge \$1 for every query to the database.

You want to figure out the *median* household income—that is, the n^{th} household income in the combined, sorted list. You must write a recursive algorithm.

Some hints:

- If you want to write pseudocode, you’ll probably want to use a function to query the database. If it helps, you can assume that the function `query( $D_i$ ,  $k$ )` returns you the k th smallest element from database i in constant time. So you will want to make calls like `query( $D_1$ ,  $n/2$ )` to indicate that you’re querying database 1 with the value $n/2$.
- If you like, you can assume that n is a power of two. (This will mean that if you divide n in half, you’ll always get a whole number.)
- Regardless of whether you assume that the lists have even length, you will need to deal with medians of even-length lists, since the overall set of numbers you are finding the median of is certainly even (since it’s $2n$). Traditionally, these are defined as the average of the $\lceil n/2 \rceil$ th and $\lfloor n/2 \rfloor$ th items, but you can just choose one to define as the median. For example, choose the $\lceil n/2 \rceil$ th item to be the median, meaning that in a list containing 1, 3, 5, 10, the median would be 5.
- Finally, a tricky thing about this problem is that you can’t manipulate the databases, you can only query them. In particular, you can’t delete half of the database and then pass the remaining half to your next recursive call. Instead, you’ll have to pass around indices into the databases.